

EXERCICE – EXPLICATION DE CODE ET UTILISATION DES HOOKS

REPONSE AU QUESTION PARTI 1

1. Expliquez le fonctionnement du code

. Le rôle de useState Dans Counter

```
const [count, setCount] = useState(0);
```

Ici useState sert à stocker la valeur du compteur

Count contient la valeur actuelle et setCount permet de la modifier

Chaque fois que tu appelles setCount React ré-exécute le composant

Pour afficher la nouvelle valeur.

DANS APP

```
const [status, setStatus] = useState(true);
```

Dans cette fonction l'état sert juste à savoir si on doit afficher ou cache le composant counter si status est true le composant est rendu et si c'est false il disparaît de la page

2. À quoi sert useEffect dans Counter ?

```
useEffect(() => {  
  console.log("MONTAGE / MISE À JOUR :", count);  
  return () => {  
    console.log("NETTOYAGE :", count);  
  };  
});
```

Au montage, il affiche : MONTAGE / MISE À JOUR 0

Il prépare aussi une **fonction de nettoyage** qui sera exécutée juste avant le prochain rendu ou au démontage

3. Quand la fonction de nettoyage est-elle appelée ?

LA FONCTION est appelle

```
() => {  
  console.log(return "NETTOYAGE :", count);  
};
```

Quand la fonction est appelée Avant chaque mise à jour du composant si count passe de 1 à 2, alors je pense que React nettoie d'abord l'ancien effet

Quand le composant est retiré du DOM c'est ce qui arrive quand on clique sur "Changer le statut" pour cacher <Counter />

4. Que se passe-t-il quand on clique sur "Changer le statut" ?

Bouton dans App :

```
onClick={() => setStatus(!status)}
```

Le bouton sert à afficher ou retirer le composant Counter du DOM

5. Quelle est la différence entre le montage, la mise à jour, et le démontage du composant Counter ?

Montage

C'est la première fois que React affiche le composant **UseEffect** vas s'exécute juste après l'affichage

Mise à jour

Cela arrive quand un état change `setCount`
Le composant est ré-affiché avec les nouvelles données

`useEffect` se relance

La fonction de nettoyage de l'ancien effet est appelée juste avant

Démontage

Le composant est retiré de la page quand `status` devient `false` React appelle la fonction de nettoyage une dernière fois

PARTIE 2 Exemple simple avec `useReducer`

Donnez un exemple simple d'un composant React utilisant le hook `useReducer` pour gérer un compteur

```

CounterReducer.jsx > CounterReducer
1  import React, { useReducer } from "react";
2
3  function reducer(state, action) {
4    switch (action.type) {
5
6      case "increment":
7        return { count: state.count + 1 };
8
9      case "decrement":
10       return { count: state.count - 1 };
11
12     case "reset":
13       return { count: 0 };
14
15     default:
16       return state;
17   }

```

```

CounterReducer.jsx > CounterReducer
18
19
20 export default function CounterReducer() {
21
22   const [state, dispatch] = useReducer(reducer, { count: 0 });
23
24   return (
25     <div className="p-4 bg-slate-800 rounded-lg text-white text-center">
26
27       <p className="text-xl mb-4">Compteur : {state.count}</p>
28
29       <button
30         onClick={() => dispatch({ type: "increment" })}
31         className="bg-blue-600 hover:bg-blue-700 px-4 py-2 rounded mr-2"
32       >
33         +1
34       </button>
35
36       <button
37         onClick={() => dispatch({ type: "decrement" })}
38         className="bg-red-600 hover:bg-red-700 px-4 py-2 rounded mr-2"
39       >
40         -1
41       </button>
42
43       <button
44         onClick={() => dispatch({ type: "reset" })}
45         className="bg-gray-600 hover:bg-gray-700 px-4 py-2 rounded"
46       >
47         Reset
48       </button>
49     </div>
50   );
51 }
52
Ln 34, Col 16  Spaces: 4  UTF-8  CRLF  JavaScript JSX  Port: 5500  Quokka
02:35
09/11/2025

```